

Diagnosing Compositional Generalization in Sequential Robot Tasks

Yixiao Wang, Cheng-En Wu, Lingfeng Sun, Pengcheng Wang, Xiang Ji,
Boyuan Liang, Guojian Zhan, Masayoshi Tomizuka

University of California, Berkeley

Abstract: Sequential robot manipulation requires policies to execute novel combinations of familiar instruction components. However, collecting demonstrations for all possible instruction tuples is combinatorially expensive, while sparsely covered datasets often fail under out-of-distribution recombination. This paper studies compositional generalization through the lens of instruction-space coverage. We decompose the generalization gap into three sources: *marginal instruction shift*, *instruction-compositional shift*, and *context-action shift*. This decomposition allows us to diagnose when sparse training coverage is sufficient, and what structure the training set must preserve for reliable action prediction. Our results show that exhaustive tuple enumeration is unnecessary: a structured subset, as small as one quarter of the full task space, can recover strong out-of-distribution performance when it covers action-relevant dependencies. We further find that sparse training often fails due to instruction steering rather than missing low-level skills; finetuning only one demonstration per task improves OOD success from 0.4% to 54.7%. For semantically dependent tasks, effective coverage must capture relational structure rather than only factor diversity. These findings suggest that efficient robot data collection should prioritize dependency coverage in instruction space over exhaustive task expansion. More results are available in the supplementary material. Project website: https://yixiaowang7.github.io/Diagnosing_Compositional_Generalization_Robot_Page/.

Keywords: Compositional Generalization; Data-Efficient Imitation Learning

1 Introduction

Sequential robot manipulation often requires executing a *composite* instruction composed of multiple subtasks. A robot may need to pick a specified object, place it into a specified container, and then interact with another target. Existing approaches typically handle such problems in two ways. One family of methods uses a hierarchical pipeline, where a high-level controller selects the active subtask and a low-level policy executes the corresponding atomic instruction [1, 2, 3]. Another family trains end-to-end instruction-conditioned policies that map observations and the full instruction directly to actions [4, 5]. Both paradigms can perform well when training and test instructions substantially overlap. However, real deployments require *systematic recombination*: the robot must execute new instruction tuples whose individual components are familiar, but whose combinations were rarely or never observed during training.

A naive response is to collect demonstrations for every possible instruction tuple. This strategy is usually impractical, because the number of task combinations grows combinatorially with the number of instruction factors. In practice, robot datasets are often *compositionally sparse*: they cover individual objects, targets, and primitive behaviors, but only a small fraction of their possible recombinations. This raises a fundamental question for robot data collection: *what coverage of the instruction space is actually necessary for compositional generalization?*

This paper studies compositional generalization from the perspective of *instruction-space coverage*. We formalize the generalization gap between a training distribution and a test distribution over held-out recombinations of familiar instruction factors. The resulting analysis decomposes the gap into three sources: *marginal instruction shift*, *instruction-compositional shift*, and *context-action shift*. This decomposition highlights a key distinction: observing every atomic instruction value is not enough if the training data do not cover the dependencies among instruction factors that determine the action distribution. Conversely, full Cartesian coverage is not always necessary when the relevant dependency structure is sufficiently covered. This perspective also clarifies the role of stage-wise modularity. A modular policy that conditions only on the active subtask can reduce unnecessary dependence on irrelevant instruction factors, leading to a sharper stage-wise generalization bound. However, modularity alone is not a complete solution. Inactive instruction factors can influence the contexts, and it requires a high-level planner that decomposes the task and generates dependency-preserving subtask instructions. Thus, the central issue is not only how the policy is parameterized, but also whether the training support identifies the instruction dependencies that govern behavior.

Guided by this analysis, we conduct controlled sequential robot manipulation experiments with both independent and semantically dependent instruction structures. The experiments provide three main insights. First, exhaustive enumeration of all task tuples is unnecessary; a carefully structured subset, even one quarter of the full task space, can be sufficient when it covers the dependencies that matter for action prediction. Second, sparse training often fails not because the policy lacks reusable subtask skills, but because it cannot correctly steer those skills under unseen instruction recombinations. Finetuning with only one demonstration per task improves OOD success from 0.4% to 54.7%, supporting this interpretation. Third, when task factors are semantically dependent, effective coverage must capture the relational structure that determines the correct action, rather than only ensuring factor diversity. Together, these results suggest a practical principle for robot data collection: demonstrations should be allocated to cover action-relevant dependency structure instead of exhaustively expanding the task set.

2 Related Work

Language-Conditioned Robot Policies. Recent robot learning systems train generalist policies that map observations and language or multimodal prompts to actions [6, 4, 7, 2, 5]. These policies typically use Transformer backbones [8], with action heads ranging from autoregressive token prediction [9] to diffusion- or flow-based generation [10, 2]. Multimodal systems such as VIMA and Interleave-VLA further show how multimodal tokens can specify diverse manipulation tasks under a unified interface [11, 5]. While these works demonstrate scalable instruction-conditioned imitation learning, our question is more specific: when the instruction space is combinatorially large, which instruction tuples must be covered to generalize to held-out recombinations?

Compositional Generalization in Language and Robotics. Systematic generalization has long been studied in language and sequence modeling, where models can perform well on random splits but fail on recombinations of familiar primitives [12, 13, 14]. Robotics adds difficulty because the instruction tuple also changes the induced context and action distribution. Benchmarks such as VIMA-Bench, ClevrSkills, and RoboHiMan evaluate held-out combinations of objects, attributes, skills, or long-horizon task structures [11, 15, 16]. Our work studies the data-side mechanism behind these failures: we decompose the generalization gap into *marginal instruction shift*, *instruction-compositional shift*, and *context-action shift*, and test how training support controls these terms.

Data Coverage and Combinatorial Designs. Recent work reduces robot data cost by scaling demonstrations, generating trajectories, or varying environmental factors. MimicGen increase data coverage by object-centric subtask motion transformation [17]. Closest to our motivation, Gao et al. [18] study efficient data collection by heuristically varying environmental factors. We instead study sequential instruction factors and ask how to design training coverage. This connects to design-

of-experiments and combinatorial testing, where orthogonal and covering arrays cover low-order factor interactions without enumerating the full Cartesian product [19, 20, 21].

Modularity and Hierarchy. Modularity and hierarchy are long-standing inductive biases for long-horizon control, spanning options and hierarchical reinforcement learning [22, 23], task-and-motion planning [24], and language-guided planner-controller systems [25, 2, 26, 3]. These systems can reduce coupling between inactive instructions and current actions by exposing an active stage or subgoal. Our analysis formalizes this benefit through a stage-wise modular policy, which removes the explicit instruction-compositional term from the gap. However, modularity alone is insufficient: inactive instructions can still determine future contexts, and dependent tasks require the planner or dataset to communicate cross-stage relations. Our contribution is therefore a data coverage principle for deciding when sparse instruction data is enough and when it leaves systematic relational errors.

3 Problem Statement

We formalize sequential robot tasks as the execution of an ordered n -tuple of discrete subtask indices $l = (l_1, \dots, l_n) \in \mathcal{L}_1 \times \dots \times \mathcal{L}_n := \mathcal{L}$, where each $\mathcal{L}_j = \{0, \dots, m\}$. Let $z \in \mathcal{Z}$ denote the time- t context (e.g., observation and robot state) and $a \in \mathcal{A}$ be the robot action. Robot policy is modeled as a conditional action distribution, $p(a \mid z, l_1, \dots, l_n)$. The execution is naturally decomposed into stages governed by a stage indicator $\sigma : \mathcal{Z} \rightarrow \{1, \dots, n\}$, such that $\sigma(z) = i$ signifies that subtask l_i is currently active.

3.1 Compositional Generalization Gap

We assume the training data are sampled from a joint distribution $p(z, a, l)$, where $l = (l_1, \dots, l_n) \in \mathcal{L}_1 \times \dots \times \mathcal{L}_n$ denotes a compositional instruction. The training support is

$$\begin{aligned} E_{\text{train}} &:= \text{supp}(p(l)) \\ &= \{l \in \mathcal{L}_1 \times \dots \times \mathcal{L}_n \mid p(l) > 0\}. \end{aligned} \quad (1)$$

To model compositional generalization, we introduce a test-time joint distribution $q(l)$ over valid recombinations of familiar factors, with support $E_{\text{total}} := \text{supp}(q)$. The OOD set is then

$$E_{\text{ood}} := E_{\text{total}} \setminus E_{\text{train}}. \quad (2)$$

We define the expected risk $\mathcal{R}$ over a distribution $\mathcal{D} \in \{p, q\}$ as:

$$\mathcal{R}_{\mathcal{D}}(\theta) := \int_{\mathcal{Z} \times \mathcal{A} \times \mathcal{L}} \mathcal{D}(z, a, l) L_{\theta}(z, a, l) dz da dl. \quad (3)$$

The *compositional generalization gap* quantifies the performance degradation caused by the distribution shift from training to testing:

$$\Delta_q(\theta) := \mathcal{R}_q(\theta) - \mathcal{R}_p(\theta) = \int_{\mathcal{Z} \times \mathcal{A} \times \mathcal{L}} (q(z, a, l) - p(z, a, l)) L_{\theta}(z, a, l) dz da dl \quad (4)$$

4 Theoretical Analysis

We begin with Proposition A.1 (Proof is provided in Appendix A), which identifies three distinct sources of the compositional generalization gap. The decomposition is centered on the instruction sequence l , since the gap arises from a mismatch between the training and test instruction distributions.

Proposition 4.1 (General upper bound on the compositional generalization gap). *Assume the loss is measurable and uniformly bounded on $\text{supp}(p) \cup \text{supp}(q)$, i.e.,*

$$0 \leq L_{\theta}(z, a, l) \leq M \quad \forall (z, a, l) \in \text{supp}(p) \cup \text{supp}(q).$$

Then, for any fixed $i \in \{1, \dots, n\}$,

$$|\Delta_q(\theta)| \leq M \left(\|q(l_i) - p(l_i)\|_1 + \mathbb{E}_{l_i \sim p(l_i)} [\|q(l_{-i} | l_i) - p(l_{-i} | l_i)\|_1] \right. \\ \left. + \mathbb{E}_{l \sim p(l)} [\|q(z, a | l) - p(z, a | l)\|_1] \right). \quad (5)$$

where

$$\|\mu - \nu\|_1 := \int_{\mathcal{X}} |\mu(x) - \nu(x)| dx.$$

for two distributions μ, ν .

From Proposition A.1, bounded training loss alone does not imply good compositional generalization. Rather, a small gap requires simultaneous control of three discrepancies. The first term, $\|q(l_i) - p(l_i)\|_1$, measures *marginal shift* in the subtask instruction l_i , which is unavoidable without broader training coverage. The second term, $\mathbb{E}_{l_i \sim p(l_i)} [\|q(l_{-i} | l_i) - p(l_{-i} | l_i)\|_1]$, captures *instruction compositional shift*, namely whether familiar subtasks are recombined in unfamiliar ways. The third term, $\mathbb{E}_{l \sim p(l)} [\|q(z, a | l) - p(z, a | l)\|_1]$, measures *context-action shift* conditioned on the full instruction sequence caused by previous term. The latter two terms are amplified when the policy entangles the active subtask with irrelevant inactive instructions. Hence, once marginal coverage is ensured, compositional generalization depends primarily on how the policy uses instruction structure. This motivates policy which can isolate stage-relevant instruction information while suppressing unnecessary dependence on l_{-i} .

4.1 Stage-wise Modular Policy

The preceding analysis shows that compositional generalization depends critically on instruction structure. This motivates the hierarchical parameterization adopted in systems such as SayCan [1], $\pi_{0.5}$ [2], and Gemini Robotics [3], which we term a *stage-wise modular policy*. Under this formulation, a high-level oracle identifies the active stage, and a low-level controller conditions only on the current context and the corresponding subtask instruction. Formally, let $\sigma : \mathcal{Z} \rightarrow \{1, \dots, n\}$ denote the stage indicator. Then, at context z ,

$$\pi_\theta(a | z, l) = \pi_{\theta, \sigma(z)}(a | z, l_{\sigma(z)}).$$

This modularization isolates the active subtask and leads to a sharper stage-wise generalization bound.

Under this modular parameterization, the loss can be written in stage-wise form as

$$L_\theta(z, a, l) = \sum_{i=1}^n \mathbf{1}\{\sigma(z) = i\} \ell_{i, \theta}(z, a, l_i),$$

where $\ell_{i, \theta}(z, a, l_i)$ denotes the loss incurred at stage i and depends only on the active subtask l_i .

Corollary 4.2 (Bound for stage-wise modular policy). *For a stage-wise modular policy, we assume that for each stage i ,*

$$0 \leq \ell_{i, \theta}(z, a, l_i) \leq M_i \quad \forall (z, a, l_i) \in \text{supp}(p_i) \cup \text{supp}(q_i).$$

where $q_i = \int q(z, a, l) dl_{-i}$ and $p_i = \int p(z, a, l) dl_{-i}$. Then

$$|\Delta_q^m(\theta)| \leq \sum_{i=1}^n M_i \left(\|q(l_i) - p(l_i)\|_1 + \mathbb{E}_{l_i \sim p} \|q(z, a | l_i) - p(z, a | l_i)\|_1 \right). \quad (6)$$

Corollary A.2 (proof is provided in Appendix A) shows that stage-wise modularity removes the explicit *instruction compositional shift* and yields a simpler stage-wise training loss. This simplification depends critically on access to such an indicator, or on learning it accurately. More fundamentally, removing l_{-i} from the policy input does not remove its influence on the induced context-action distribution $p(z, a | l_i)$. Unseen values of l_{-i} can still generate novel contexts, under which the policy still fail to generalize. Moreover, such modularization may induce an information bottleneck: if the optimal action depends on instruction factors outside l_i , then $p(a | z, l_i)$ is inherently ambiguous.

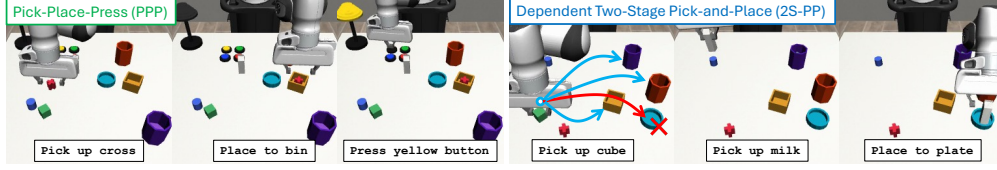


Figure 1: Illustration of Pick-Place-Press and Dependent Two-Stage Pick-and-Place tasks.

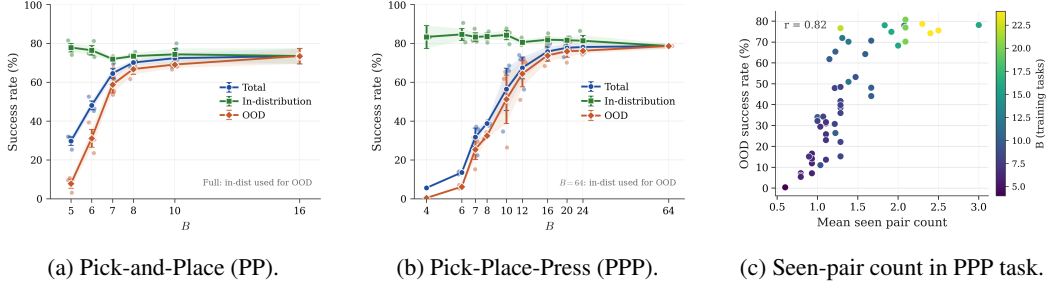


Figure 2: Full tuple coverage is unnecessary. Higher seen pair count predicts stronger OOD success.

5 Experiments

Our experiments study the following question: *what coverage of the instruction space is necessary for compositional generalization in sequential robot tasks?* Collecting demonstrations for every instruction tuple is sufficient but scales poorly, as the number of tuples grows exponentially with the number of subtasks. Proposition A.1 suggests a theoretical view: since the generalization gap is bounded by marginal, instruction-compositional, and context-action shifts, controlling these terms should matter more than exhaustive enumeration. Guided by this decomposition, we study three questions: *Q1: Is full tuple coverage necessary? Q2: What is missing when sparse coverage fails? Q3: What coverage is needed for dependent instructions?*

5.1 Experimental Setup

Task Design. We design three tasks with increasing compositional complexity in robomimic [27].

1. **Pick-and-Place (PP).** The instruction is $l = (o, c)$, where $o \in \{1, \dots, 4\}$ is the object and $c \in \{1, \dots, 4\}$ is the container. The robot picks object o and places it into container c . The full task space contains $|E_{\text{total}}| = 4 \times 4 = 16$ tasks.
2. **Pick-Place-Press (PPP).** The instruction is $l = (o, c, b)$, where $b \in \{1, \dots, 4\}$ denotes the button color. The robot places object o into container c , then presses button b , activating the lamp with the corresponding color. The full task space contains $|E_{\text{total}}| = 4 \times 4 \times 4 = 64$ tasks. The three subtasks are independent.
3. **Dependent Two-Stage Pick-and-Place (2S-PP).** The full task is $l = (o_1, c_1, o_2, c_2)$, with constraints $o_1 \neq o_2$ and $c_1 \neq c_2$. The robot first places o_1 into c_1 , then places o_2 into c_2 . The full task space contains $|E_{\text{total}}| = 4 \times 4 \times 3 \times 3 = 144$ tasks. **However, the policy only receives the partial instruction $l = (o_1, o_2, c_2)$.** Thus, it must infer the missing container c_1 for o_1 while satisfying $c_1 \neq c_2$, making the task dependent and more challenging.

Data and Policy. We use a customized motion planner to generate 500, 300, and 100 demonstrations per task for PP, PPP, and 2S-PP, yielding 8,000, 19,200, and 14,400 demonstrations in total, respectively. All experiments use the same transformer-based behavior cloning policy with identical observation and action spaces. At each time step, the policy receives privileged state information, including the 6D gripper pose, and visual observations from third-person and eye-in-hand cameras. The action space consists of end-effector position and orientation deltas, together with gripper ac-

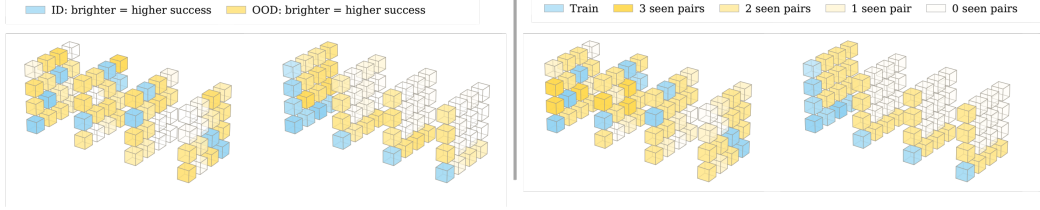


Figure 4: Representative examples showing that OOD tasks with higher seen-pair coverage tend to achieve higher success rates. More results are shown in Appendix C.

tuation. We use DINOv2 [28] as the vision encoder and a transformer-based flow-matching policy. Further details are provided in Appendix B.

Evaluation Metrics. We report three success-rate metrics: **Total SR**, computed over all tasks in E_{total} ; **ID SR**, computed over training tasks E_{train} ; and **OOD SR**, computed over held-out recombinations $E_{\text{ood}} = E_{\text{total}} \setminus E_{\text{train}}$. Unless otherwise noted, each evaluation pools results over three training seeds and 30 trials per task. For 2S-PP, we also report the **same-container violation rate**, measuring failures that violate the cross-stage container constraint after the first placement.

5.2 Q1: Is Full Tuple Coverage Necessary?

We first test whether full Cartesian coverage is required for compositional generalization. For each training budget $B = |E_{\text{train}}|$, we sample training tasks while ensuring that every subtask instruction value appears at least once. This controls the *marginal shift term* $\|q(l_i) - p(l_i)\|_1$ in Proposition A.1. Therefore, poor OOD SR cannot be attributed merely to unseen subtask instructions.

Figure 2 shows that small training sets still yield poor OOD success despite marginal coverage. However, full tuple coverage is also unnecessary. PP saturates around $B = 8$, while PPP saturates around $B = 16$, far below the full 16- and 64-task Cartesian products, but achieving similar success. Another finding is that when B is large enough, low training loss may generalize to unseen instruction tuples and their induced contexts, and the *context-action shift* appears insignificant; both phenomena are likely because the policy learns to focus on subtask-relevant information.

To identify the relevant structure, we analyze the PPP task at the level of instruction pairs. For each held-out task $l = (o, c, b)$, we define its *seen pair count* as the number of observed pairs among (o, c) , (o, b) , and (c, b) . From Figure 2c, OOD success increases with this count. Figure 4 illustrates representative examples where OOD task success correlates strongly with the number of seen pairs. These results also support Proposition A.1: pairwise coverage mitigates the instruction-compositional discrepancy $q(l_{-i} | l_i) - p(l_{-i} | l_i)$ and consequently reduces the induced context-action shift.

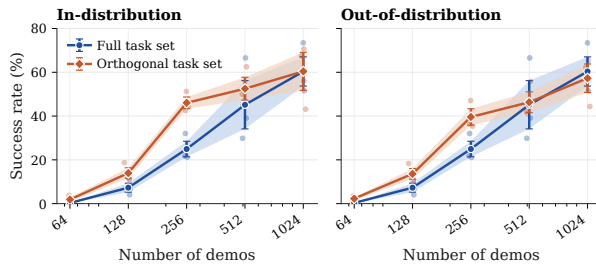


Figure 3: Orthogonal set is more effective than full-set training when demonstrations are limited.

This observation motivates an instruction-coverage principle: greater pairwise coverage in the training set improves OOD generalization. Accordingly, the orthogonal 16-task design, inspired by design-of-experiments theory [19, 20], ensures that every pairwise combination of instruction factors appears in training. Experiments show that, with only one quarter of the full 64-task space, it preserves the relevant pairwise coverage and achieves 78.2% OOD success, close to full-task training. Additional results, including 53 different PPP training-set configurations, are provided in Appendix C.

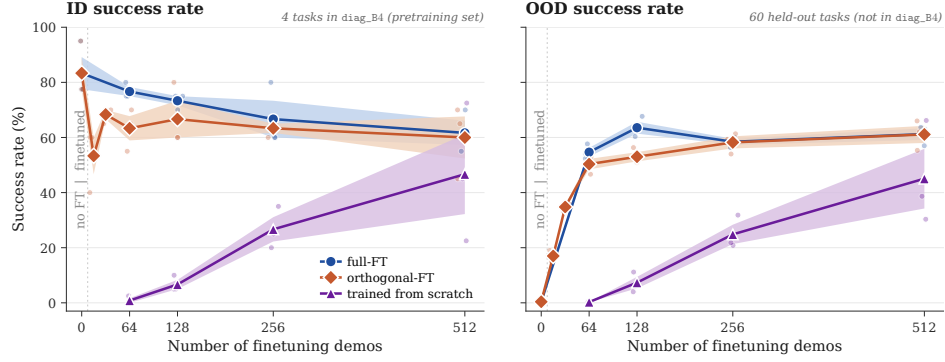


Figure 5: Finetuning from sparse diagonal pretraining. Small amounts of broad-coverage data substantially improve OOD success, suggesting that sparse pretraining learns reusable skills but not reliable instruction steering, which can be acquired with limited additional data.

Fixed-budget comparison. The same principle has a practical implication for data collection. Under a fixed demonstration budget, exhaustive task coverage allocates few demonstrations to each tuple, whereas an orthogonal task set concentrates demonstrations on structurally informative combinations. As shown in Figure 3, orthogonal-set training performs better at low and medium budgets, and the gap narrows only when the total number of demonstrations becomes large. Therefore, avoiding full enumeration can improve both task efficiency and sample efficiency.

5.3 Q2: What Is Missing When Sparse Coverage Fails?

The preceding experiments show that sparse task sets can fail even when all subtask instruction values are observed. Proposition A.1 points to two remaining sources of failure: *instruction-compositional shift* and *context-action shift*. This raises an operational question: does the policy fail from lack of subtask skills, or from poor skill routing under unseen instruction combinations?

To separate these hypotheses, we choose Pick-Place-Press (PPP) task and pretrain on diagonal sets, i.e., the minimal task set $(o, c, b) \in (0, 0, 0), (1, 1, 1), (2, 2, 2), (3, 3, 3)$, which covers all instruction marginals, and then finetune with a small number of demonstrations from broader task sets. Figure 5 shows that diagonal pretraining achieves high ID success but nearly zero OOD success. However, with only *one* demonstration per full-set task, finetuning raises OOD success from 0.4% to 54.7%, whereas scratch training with the same total budget remains substantially weaker. ID success decreases after finetuning, suggesting a trade-off induced by distributional reweighting rather than a loss of subtask competence. Finetuning on the orthogonal set shows a similar trend. We also evaluate pretraining on six-task sets and observe similar conclusions. Further analysis is provided in Appendix D.

These results indicate that sparse pretraining learns reusable subtask skills, but not the mapping from unseen instruction tuples to the appropriate behavior. That means sparse coverage pretraining leaves large instruction-compositional discrepancies; a small amount of broad-coverage finetuning can substantially reduce this gap.

5.4 Q3: What Coverage Is Needed for Dependent Instructions?

Section 5.2 shows that high pairwise coverage in the training task set leads to strong OOD performance. We next ask when such coverage is necessary and whether it should apply to all instruction pairs.

We instantiate this setting on Dependent Two-Stage Pick-and-Place (2S-PP). The robot must place two objects into different containers, while the policy observes only $l = (o_1, o_2, c_2)$. Hence, the first placement is not independently specified; it must be selected in relation to the second placement. We use four values of $o_1 \in \{1, 2, 3, 4\}$ and three values of $c_2 \in \{1, 2, 3\}$. The full task set contains $4 \times 3 \times 3 = 36$ tasks. Starting from the orthogonal design (12 training tasks) with all 12 dependent

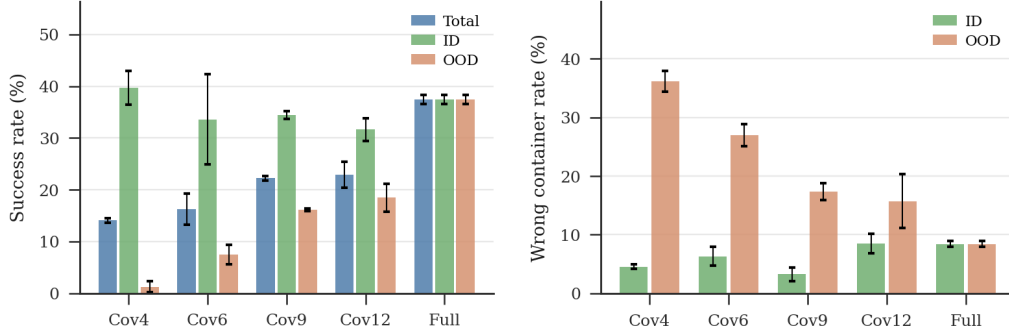


Figure 6: Performance and same-container violations in 2S-PP. Broader dependent-pair coverage improves success and reduces relational errors (e.g., put object o_1 into container c_2).

pairs (o_1, c_2) covered (Cov12), we reduce the number of observed dependent pairs to 9, 6, and 4, yielding Cov9, Cov6, and Cov4, respectively, while keeping 12 training tasks.

We define the same-container violation rate as the probability that the policy places o_1 into c_2 , conditioned on a successful first placement. Figure 6 shows that broader dependent-pair coverage improves OOD success and reduces same-container violations. This suggests that, when instruction factors are semantically dependent, their joint combinations should be more densely covered in the training set. Notably, fewer observed (o_1, c_2) pairs yield higher ID success and a lower ID same-container violation rate. We attribute this to denser supervision per ID pair: with fewer observed (o_1, c_2) pairs, each pair receives more demonstrations.

These results further reveal the mechanism behind the benefit of pairwise coverage. The required coverage is not arbitrary pairwise diversity, but coverage of the dependent instruction factors. This also explains why a purely stage-wise modular policy can be insufficient unless the high-level planner explicitly communicates the relational constraint to the low-level controller. Further analysis, including failure-case analysis, is provided in Appendix E.

6 Conclusion and Limitation

We presented a theory-driven study of compositional generalization in sequential robot tasks. Our analysis decomposes the generalization gap into *marginal instruction shift*, *instruction-compositional shift*, and *context-action shift*. Guided by this decomposition, our experiments provide three main insights. First, exhaustive enumeration of all task tuples is unnecessary; a carefully structured subset, even one quarter of the full task space, can be sufficient when it covers the dependencies that matter for action prediction. Second, sparse training often fails not because the policy lacks subtask skills, but because it cannot correctly steer those skills under unseen instruction recombinations: finetuning with only one demonstration per task can improve OOD success from 0.4% to 54.7%. Third, when task factors are semantically dependent, effective coverage must capture the relational structure that determines the correct action, rather than only ensuring factor diversity. Overall, these findings suggest that efficient robot data collection should prioritize dependency coverage in the instruction space instead of exhaustively expanding the task set.

This study has several limitations. First, while our experiments cover both independent and dependent instructions, they are restricted to a small set of sequential manipulation tasks, as the required demonstrations and evaluation tasks grow exponentially with instruction complexity; extending this analysis to large-scale training, and characterizing how such coverage patterns inform pretraining, is an important direction for future work. Second, our analysis centers on policies trained from scratch atop a frozen vision encoder; a natural extension is to pretrained vision-language-action (VLA) models, both to diagnose the origins of compositional generalization and to assess how our coverage principle transfers to this setting.

Acknowledgments

If a paper is accepted, the final camera-ready version will (and probably should) include acknowledgments. All acknowledgments go at the end of the paper, including thanks to reviewers who gave useful comments, to colleagues who contributed to the ideas, and to funding agencies and corporate sponsors that provided financial support.

References

- [1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng. Do as i can, not as i say: Grounding language in robotic affordances, 2022. URL <https://arxiv.org/abs/2204.01691>.
- [2] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [3] G. R. Team, A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.
- [4] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [5] C. Fan, X. Jia, Y. Sun, Y. Wang, J. Wei, Z. Gong, X. Zhao, M. Tomizuka, X. Yang, J. Yan, et al. Interleave-vla: Enhancing robot manipulation with interleaved image-text instructions. *arXiv preprint arXiv:2505.02152*, 2025.
- [6] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [7] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [9] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [10] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [11] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan. Vima: General robot manipulation with multimodal prompts, 2023. URL <https://arxiv.org/abs/2210.03094>.

- [12] B. Lake and M. Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International conference on machine learning*, pages 2873–2882. PMLR, 2018.
- [13] B. Lake and M. Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International conference on machine learning*, pages 2873–2882. PMLR, 2018.
- [14] S. Yagcioglu, O. B. Ince, A. Erdem, E. Erdem, D. Elliott, and D. Yuret. Sequential compositional generalization in multimodal models. *arXiv preprint arXiv:2404.12013*, 2024.
- [15] S. Haresh, D. Dijkman, A. Bhattacharyya, and R. Memisevic. Clevrskills: Compositional language and visual reasoning in robotics. *Advances in Neural Information Processing Systems*, 37:38235–38266, 2024.
- [16] Y. Chen, Z. Chen, N. T. Chan, J. Chen, J. Yin, J. Shi, Y. Gao, Y.-L. Li, and J. Huo. Robohiman: A hierarchical evaluation paradigm for compositional generalization in long-horizon manipulation. *arXiv preprint arXiv:2510.13149*, 2025.
- [17] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. *arXiv preprint arXiv:2310.17596*, 2023.
- [18] J. Gao, A. Xie, T. Xiao, C. Finn, and D. Sadigh. Efficient data collection for robotic manipulation via compositional generalization. *arXiv preprint arXiv:2403.05110*, 2024.
- [19] R. N. Kacker, E. S. Lagergren, and J. J. Filliben. Taguchi’s orthogonal arrays are classical designs of experiments. *Journal of research of the National Institute of Standards and Technology*, 96(5):577, 1991.
- [20] A. S. Hedayat, N. J. A. Sloane, and J. Stufken. *Orthogonal arrays: theory and applications*. Springer Science & Business Media, 2012.
- [21] D. R. Kuhn, R. N. Kacker, and Y. Lei. *Introduction to combinatorial testing*. CRC press, 2013.
- [22] P.-L. Bacon, J. Harb, and D. Precup. The option-critic architecture. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.
- [23] O. Nachum, S. S. Gu, H. Lee, and S. Levine. Data-efficient hierarchical reinforcement learning. *Advances in neural information processing systems*, 31, 2018.
- [24] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 4(1):265–293, 2021.
- [25] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [26] L. X. Shi, M. R. Equi, L. Ke, K. Pertsch, Q. Vuong, J. Tanner, A. Walling, H. Wang, N. Fusai, A. Li-Bell, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In *Forty-second International Conference on Machine Learning*.
- [27] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2021.
- [28] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research Journal*, pages 1–31, 2024.

- [29] J. Li, D. Li, S. Savarese, and S. Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.

A Theoretical Analysis

Proposition A.1 (General upper bound on the compositional generalization gap). *Assume the loss is measurable and uniformly bounded on $\text{supp}(p) \cup \text{supp}(q)$, i.e.,*

$$0 \leq L_\theta(z, a, l) \leq M \quad \forall (z, a, l) \in \text{supp}(p) \cup \text{supp}(q).$$

Then, for any fixed $i \in \{1, \dots, n\}$,

$$\begin{aligned} |\Delta_q(\theta)| &\leq M \left(\|q(l_i) - p(l_i)\|_1 \right. \\ &\quad + \mathbb{E}_{l_i \sim p(l_i)} [\|q(l_{-i} | l_i) - p(l_{-i} | l_i)\|_1] \\ &\quad \left. + \mathbb{E}_{l \sim p(l)} [\|q(z, a | l) - p(z, a | l)\|_1] \right). \end{aligned} \quad (7)$$

Proof. Starting from

$$\Delta_q(\theta) = \int_{\mathcal{Z} \times \mathcal{A} \times \mathcal{L}} (q(z, a, l) - p(z, a, l)) L_\theta(z, a, l) dz da dl,$$

and using the factorization $l = (l_i, l_{-i})$, we have

$$q(z, a, l) - p(z, a, l) = q(l_i)q(l_{-i} | l_i)q(z, a | l) - p(l_i)p(l_{-i} | l_i)p(z, a | l).$$

Add and subtract the intermediate terms

$$p(l_i)q(l_{-i} | l_i)q(z, a | l) \quad \text{and} \quad p(l_i)p(l_{-i} | l_i)q(z, a | l),$$

to get

$$\begin{aligned} q(z, a, l) - p(z, a, l) &= q(l_i)q(l_{-i} | l_i)q(z, a | l) - p(l_i)q(l_{-i} | l_i)q(z, a | l) \\ &\quad + p(l_i)q(l_{-i} | l_i)q(z, a | l) - p(l_i)p(l_{-i} | l_i)q(z, a | l) \\ &\quad + p(l_i)p(l_{-i} | l_i)q(z, a | l) - p(l_i)p(l_{-i} | l_i)p(z, a | l) \end{aligned} \quad (8)$$

Then we have the exact decomposition

$$\Delta_q(\theta) = A_i + B_i + C_i, \quad (9)$$

where

$$\begin{aligned} A_i &= \int (q(l_i) - p(l_i)) q(l_{-i} | l_i) q(z, a | l) L_\theta(z, a, l) dz da dl, \\ B_i &= \int p(l_i) (q(l_{-i} | l_i) - p(l_{-i} | l_i)) q(z, a | l) L_\theta(z, a, l) dz da dl, \\ C_i &= \int p(l_i) p(l_{-i} | l_i) (q(z, a | l) - p(z, a | l)) L_\theta(z, a, l) dz da dl. \end{aligned}$$

Using $0 \leq L_\theta \leq M$, each term is bounded as follows:

$$|A_i| \leq M \int |q(l_i) - p(l_i)| q(l_{-i} | l_i) q(z, a | l) dz da dl = M \|q(l_i) - p(l_i)\|_1,$$

$$|B_i| \leq M \int p(l_i) |q(l_{-i} | l_i) - p(l_{-i} | l_i)| q(z, a | l) dz da dl = M \mathbb{E}_{l_i \sim p(l_i)} [\|q(l_{-i} | l_i) - p(l_{-i} | l_i)\|_1],$$

and

$$|C_i| \leq M \int p(l_i) |q(z, a | l) - p(z, a | l)| dz da dl = M \mathbb{E}_{l \sim p(l)} [\|q(z, a | l) - p(z, a | l)\|_1].$$

Combining these three inequalities with (9) yields (7). $\square$

From Proposition A.1, bounded training loss alone does not guarantee good compositional generalization. Instead, a small gap requires controlling three distinct discrepancies, each of which has a different interpretation and a different degree of controllability.

First, the marginal term $\|q(l_i) - p(l_i)\|_1$ measures whether the active subtask l_i itself is covered by the training distribution. If test-time subtasks are absent from training, then no policy can be expected to generalize reliably, since the problem is one of missing support rather than model structure. This term is therefore fundamentally a data issue: it can only be reduced by expanding the training distribution to include the relevant subtasks.

Second, the compositional term

$$\mathbb{E}_{l_i \sim p(l_i)} [\|q(l_{-i} \mid l_i) - p(l_{-i} \mid l_i)\|_1]$$

captures whether familiar subtasks are recombined with different surrounding instructions at test time. Unlike the first term, this shift arises even when each individual subtask has been observed during training. It reflects a failure to generalize across novel combinations of known factors, and thus directly characterizes the core challenge of compositional generalization.

Third, the conditional term

$$\mathbb{E}_{l \sim p(l)} [\|q(z, a \mid l) - p(z, a \mid l)\|_1]$$

measures the residual shift in context–action behavior conditioned on the full instruction sequence. In practice, this term can grow when the learned policy becomes overly sensitive to irrelevant correlations among instruction components, so that unfamiliar recombinations of otherwise familiar subtasks induce different action behavior.

This decomposition suggests that the first term is unavoidable without broader training coverage, whereas the second and third terms depend more strongly on how the policy represents and uses instruction structure. In particular, if the policy can isolate the instruction component that is relevant at the current execution stage while suppressing unnecessary dependence on inactive subtasks, then its sensitivity to novel recombinations may be reduced. This motivates introducing an idealized stage-wise modular policy as a conceptual reference point.

Corollary A.2 (Modular bound). *Assume that for each stage i ,*

$$0 \leq \ell_{i,\theta}(z, a, l_i) \leq M_i \quad \forall (z, a, l_i) \in \text{supp}(p_i) \cup \text{supp}(q_i).$$

Then

$$\begin{aligned} |\Delta_q^m(\theta)| &\leq \sum_{i=1}^n M_i \left(\|q(l_i) - p(l_i)\|_1 \right. \\ &\quad \left. + \mathbb{E}_{l_i \sim p} \|q(z, a \mid l_i) - p(z, a \mid l_i)\|_1 \right). \end{aligned} \tag{10}$$

Proof. Starting from

$$\Delta_q^m(\theta) = \int (q(z, a, l) - p(z, a, l)) L_\theta(z, a, l) dz da dl,$$

and using

$$L_\theta(z, a, l) = \sum_{i=1}^n \mathbf{1}\{\sigma(z) = i\} \ell_{i,\theta}(z, a, l_i),$$

we obtain

$$\Delta_q^m(\theta) = \sum_{i=1}^n \int \mathbf{1}\{\sigma(z) = i\} \ell_{i,\theta}(z, a, l_i) (q(z, a, l) - p(z, a, l)) dz da dl.$$

Since $\ell_{i,\theta}(z, a, l_i)$ depends only on (z, a, l_i) , we can integrate out l_{-i} and get

$$\Delta_q^m(\theta) = \sum_{i=1}^n \int \ell_{i,\theta}(z, a, l_i) (q(z, a, l_i) - p(z, a, l_i)) dz da dl_i.$$

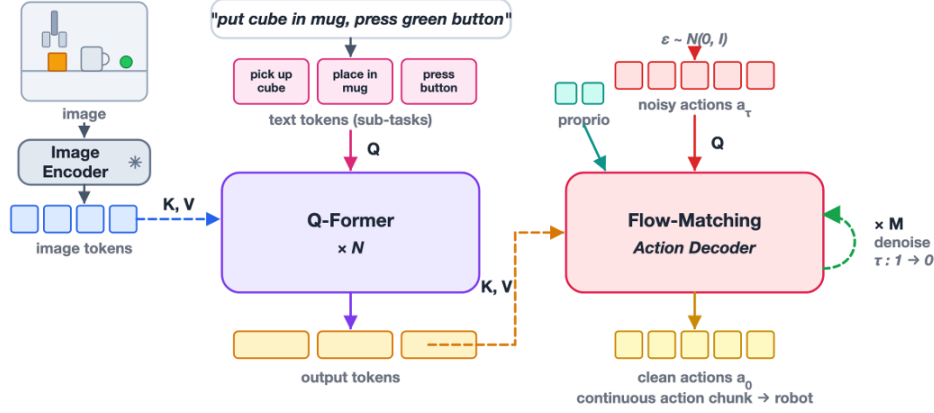


Figure 7: Policy network structure.

Then we have

$$q(z, a, l_i) - p(z, a, l_i) = (q(l_i) - p(l_i)) q(z, a | l_i) + p(l_i) (q(z, a | l_i) - p(z, a | l_i)).$$

Therefore,

$$\Delta_q^{\text{mod}}(\theta) = \sum_{i=1}^n (U_i + V_i),$$

where

$$U_i = \int \ell_{i,\theta}(z, a, l_i) (q(l_i) - p(l_i)) q(z, a | l_i) dz da dl_i,$$

and

$$V_i = \int \ell_{i,\theta}(z, a, l_i) p(l_i) (q(z, a | l_i) - p(z, a | l_i)) dz da dl_i.$$

Using $0 \leq \ell_{i,\theta} \leq M_i$, we bound

$$|U_i| \leq M_i \int |q(l_i) - p(l_i)| \left(\int q(z, a | l_i) dz da \right) dl_i = M_i \|q(l_i) - p(l_i)\|_1,$$

and

$$|V_i| \leq M_i \int p(l_i) \left(\int |q(z, a | l_i) - p(z, a | l_i)| dz da \right) dl_i = M_i \mathbb{E}_{l_i \sim p} \|q(z, a | l_i) - p(z, a | l_i)\|_1.$$

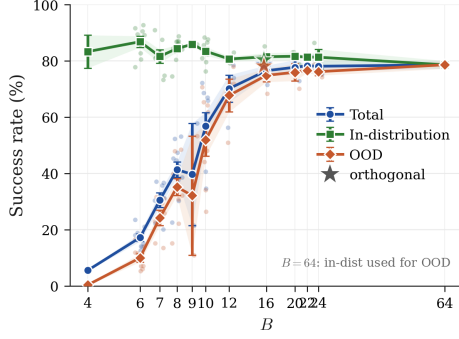
Summing over i yields (10). $\square$

B Policy Network

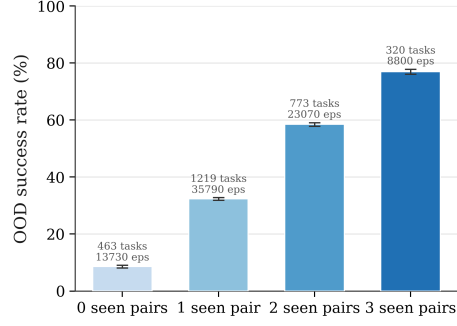
We use a frozen DINOv2 model as the image encoder. Each instruction is represented as a token, while multi-instruction inputs are represented by multiple instruction tokens that serve as queries. The image tokens serve as keys and values, following a design similar to Q-Former [29]. The resulting output tokens are then injected into the flow-matching transformer through cross-attention. Further details are shown in Figure 7.

C Q1: Is Full Tuple Coverage Necessary?

Pick-Place-Press. We trained the policy for 100k steps with a batch size of 64 on the Pick-Place-Press (PPP) task, conducting experiments across 55 different combinations in total. Figure 8a shows similar trend as Figure 2b. Figure 8b shows that OOD tasks with greater seen-pair coverage achieve higher success rates. Figures 15 and 16 show that different training sets lead to different OOD performance across tasks, and that the number of seen pairs may be correlated with OOD success. Additional results, provided in both image and HTML formats, are included in the supplementary materials.



(a) Success rate vs budget B (full experiments). The orthogonal set achieves the same performance as the full task set using only one-fourth of the tasks.



(b) OOD tasks with higher seen pairs lead to higher success.

Figure 8: Additional experiments on PPP task.

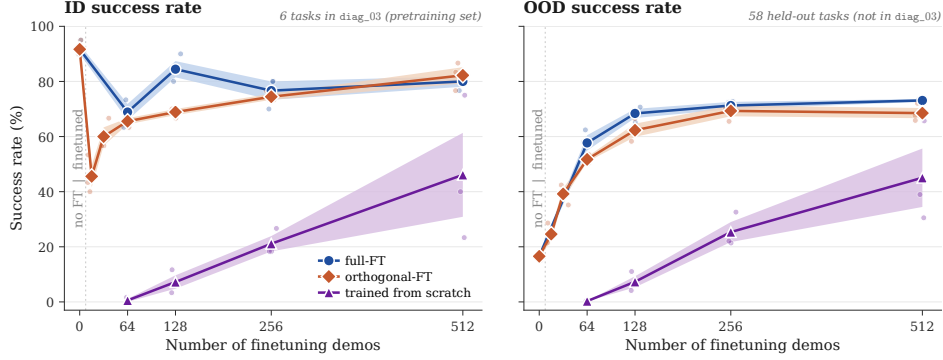


Figure 9: Finetuning from sparse pretraining (diag_03, $B = 6$). Small amounts of broad-coverage data substantially improve OOD success, suggesting that sparse pretraining learns reusable skills but not reliable instruction steering, which can be acquired with limited additional data.

D Q2: What Is Missing When Sparse Coverage Fails?

We pretrained the model for 100k steps and then finetuned it for only 5k steps, using a batch size of 64 for both stages. In addition to pretraining on the minimal diagonal sets shown in Figure 5, we also conduct experiments on another sparse-coverage set with $B = 6$, as shown in Figure 9. The results lead to a similar conclusion: finetuning with only one demonstration per task significantly improves OOD success. However, the $B = 6$ setting achieves higher OOD performance, likely because the pretrained checkpoint already exhibits stronger OOD generalization. We further conduct finetuning experiments using checkpoints pretrained with larger budgets, including $B = 8$ and $B = 16$ (orthogonal). Figure 10 shows that better pretrained checkpoints generally lead to better OOD performance after finetuning. However, when the pretrained checkpoint is already strong, as in the orthogonal setting, finetuning can degrade OOD performance. This may be because finetuning uses only

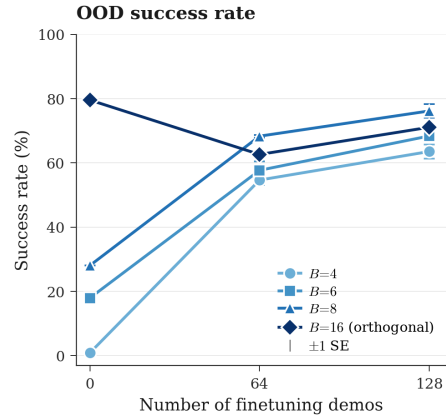


Figure 10: Finetuning from different pretrained model ($B = 4, 6, 8, 16$). **Only one demonstration** per task can significantly improve OOD success. However, finetuning can also suffer from overfitting when the pretrained model is already strong.

OOD success rate — Random coverage vs Orthogonal

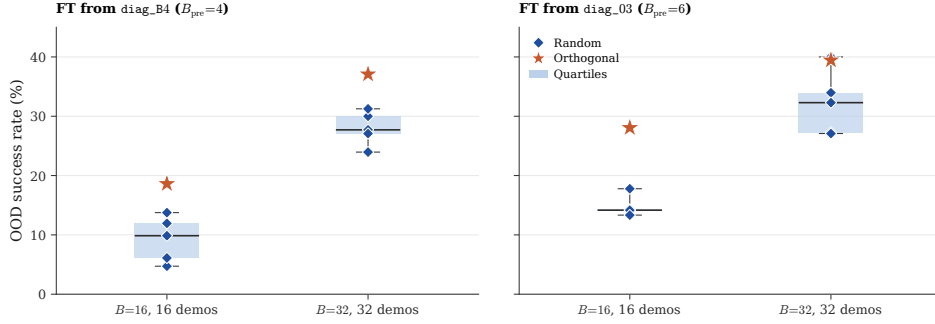


Figure 11: The orthogonal set achieves superior performance in the extremely data-limited setting.

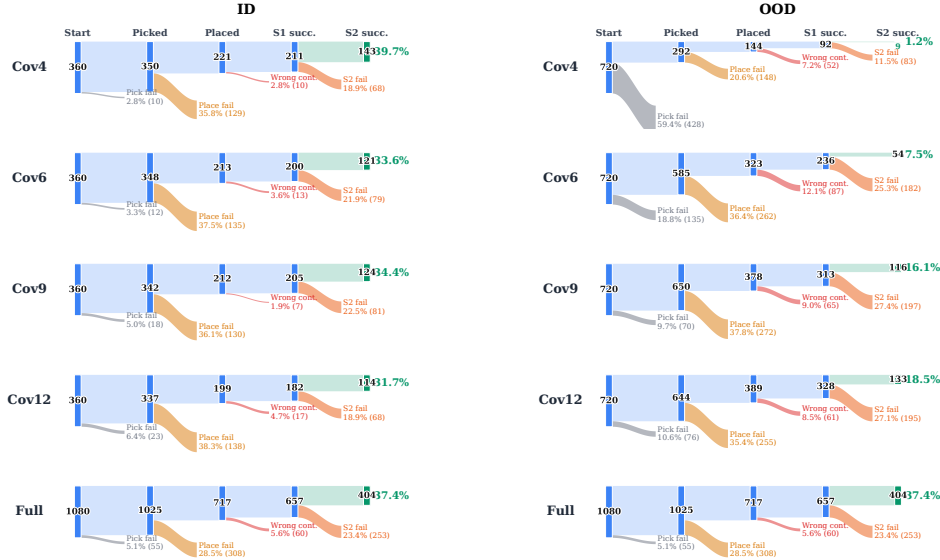


Figure 12: Failure case analysis on 2S-PP. Numbers in the figure means the number of rollouts.

64 demonstrations, one per task, which can lead to overfitting to the finetuning dataset. A similar degradation is also observed on ID tasks.

Next, we evaluate a more data-limited finetuning setting using only 16 or 32 total demonstrations, which averages to less than one demonstration per task across the full 64-task space. In this setting, we compare the orthogonal task set (comprising 2 demonstrations per task for the 32-demonstration case) against randomly sampled task sets. For the random baseline, we sample 16 or 32 distinct tasks from the 64 available tasks and collect a single demonstration for each. We generate five different training task sets via random sampling, finetune the same pretrained model on each, and compare the resulting performance to the orthogonal design. As shown in Figure 11, across two different pretrained checkpoints (diag_B4 and diag_03), finetuning on the orthogonal set substantially outperforms finetuning on randomly sampled tasks.

E Q3: What Coverage Is Needed for Dependent Instructions?

To account for the increased difficulty of this task, we trained the policy for 150k steps.

36 task set. The Sankey plot in Figure 12 shows that ID performance is similar across settings. However, for OOD tasks, performance degrades significantly when there are fewer distinct (o_1, c_2) pairs. This degradation appears not only as placing object o_1 into an incorrect container, but also as a

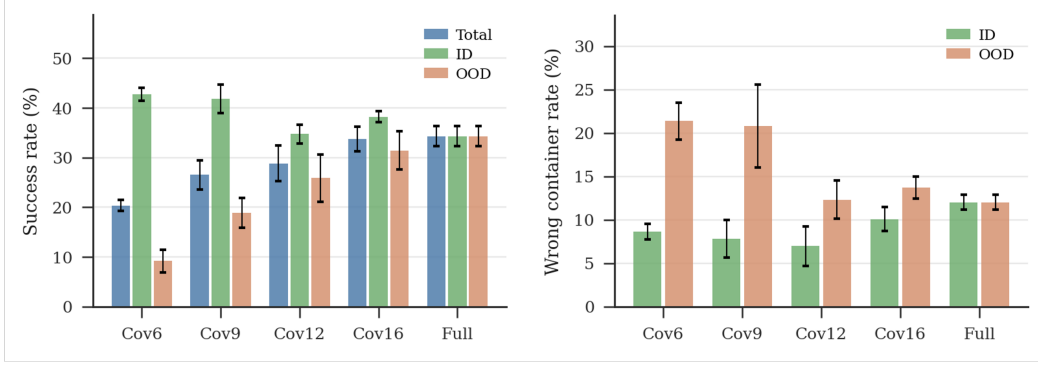


Figure 13: Performance and same-container violations in 2S-PP (48 task set). Broader dependent-pair coverage improves success and reduces relational errors (e.g., put object o_1 into container c_2).

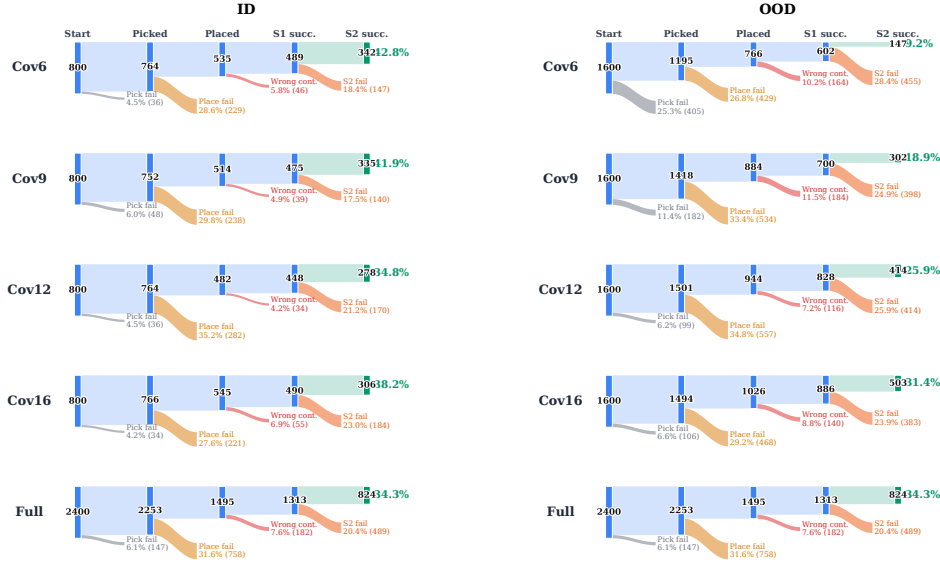


Figure 14: Failure case analysis on 2S-PP (48 task set). Numbers in the figure means the number of rollouts.

substantially lower grasp success rate. This is likely because four distinct pairs provide insufficient coverage, causing o_1 and c_2 to become spuriously correlated. This effect is quickly mitigated when the coverage is increased to six pairs, as shown in Cov6, although the wrong-container rate remains high. These results suggest that instruction dependencies require broader coverage to avoid spurious correlations.

48 task set. We extend 36 task set to 48 task set, and each for use 5 training seeds train the model and do the average. we use four values of $o_1 \in \{1, 2, 3, 4\}$ and three values of $c_2 \in \{1, 2, 3, 4\}$. The full task set contains $4 \times 3 \times 4 = 48$ tasks. Starting from the orthogonal design (16 training tasks) with all 16 dependent pairs (o_1, c_2) covered (Cov16), we reduce the number of observed dependent pairs to 12, 9, and 6, yielding Cov12, Cov9, and Cov6, respectively, while keeping 16 training tasks. Similarly, Figure 13 illustrates that broader dependent-pair coverage enhances OOD performance and minimizes same-container violations, suggesting that joint combinations of semantically dependent factors must be well-represented during training. Conversely, narrowing the coverage to fewer (o_1, c_2) pairs yields higher ID success and fewer ID violations. This ID improvement stems from denser per-pair supervision; with fewer total pairs to cover, each individual pair receives more concentrated demonstration data. Figure 14 shows the Sankey plot, which has the similar pattern as Figure 12.

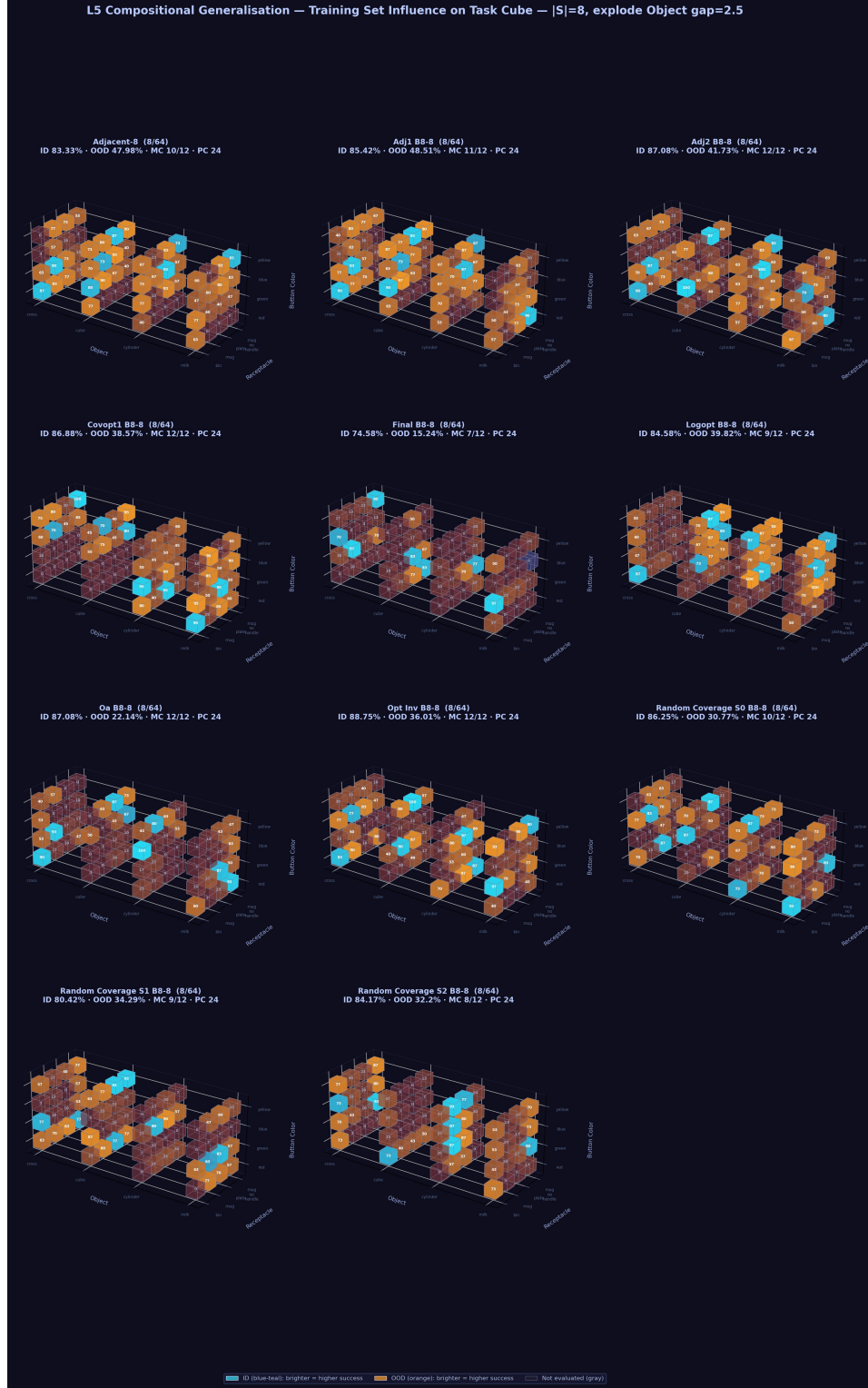


Figure 15: Success rate at $B = 8$.

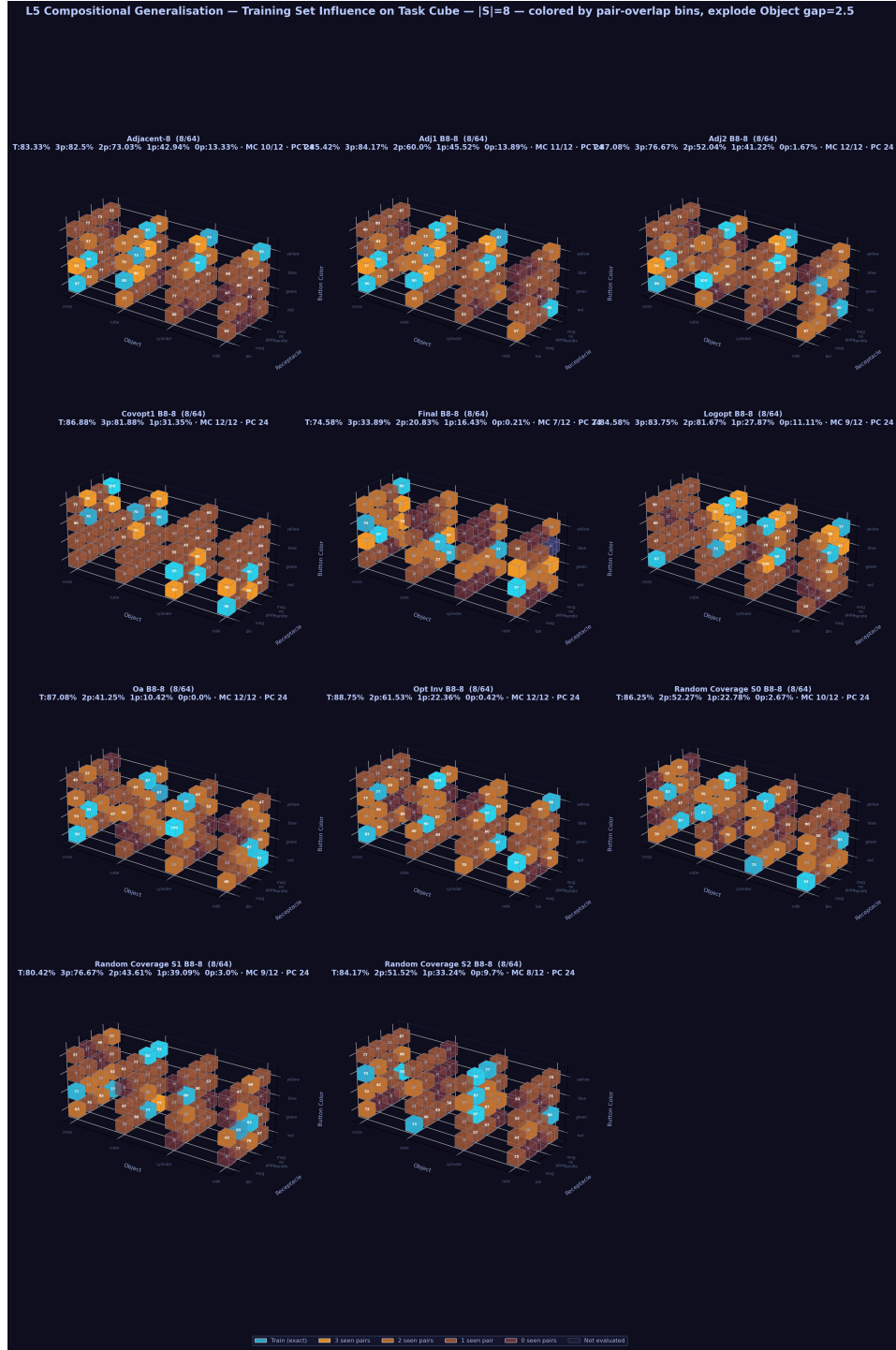


Figure 16: Seen pair count at $B = 8$.